جزوه‌ی هفته‌ی ۳: کلاس انتزاعی، dataclass، نوع‌دهی و خط لوله‌ی ترکیب‌پذیر `DataPipeline`

یک دستورالعمل تکرارپذیر برای پاک‌سازی داده‌ی حسگر

هوش مصنوعی برای مهندسی هوافضا و مکانیک (دکتری)، بخش صفر

منشأ محتوا: محتوای جدید. جزوه‌های مدرس برنامه‌نویسی شیءگرای پایتون را پوشش نمی‌دهند. مطالب پایتون با بازنویسی (نه رونویسی) از W3Schools (ماژول abc، ماژول dataclasses، ماژول typing، متدهای `__repr__()` و `__eq__()`، تکرارگرها) و GeeksforGeeks (کلاس‌های انتزاعی در پایتون، آشنایی با dataclass، جنریک‌ها در پایتون، str در برابر repr) تهیه شده است. فیلتر Hampel برگرفته از R. K. Pearson, “Outliers in process modeling and identification”, *IEEE Trans. Control Systems Technology* 10(1), 2002 و ضریب سازگاری MAD برگرفته از P. J. Rousseeuw and C. Croux, *JASA* 88(424), 1993 است.

۱. مسئله

گروه سازه بیشینه‌ی ضریب بار پرواز آزمایشی هفته‌ی ۱ را می‌خواهد. تحلیلگر A شتاب‌سنج را با میانگین متحرک ۵ نمونه‌ای هموار می‌کند و **۱٫۸۷g در ثانیه‌ی ۶۶٫۷** گزارش می‌دهد. تحلیلگر B ابتدا میانه‌ی متحرک ۷ نمونه‌ای و سپس همان میانگین را می‌گیرد و **۱٫۸۰g در ثانیه‌ی ۱۹۲** گزارش می‌دهد. هر دو اسکریپت کوتاه و معقول‌اند و هر دو بدون خطا اجرا شده‌اند.

حق با B است: ثانیه‌ی ۱۹۲ مانور کشش (pull-up) است، در حالی که ثانیه‌ی ۶۶٫۷ پرواز افقی با یک نمونه‌ی خراب است. درس مهندسی (پیش از هموارسازی جهش‌ها را حذف کنید) بخش آسان ماجراست. درس نرم‌افزاری این است که در هیچ‌کدام از دو نتیجه چیزی نمی‌گوید چگونه ساخته شده است: **ترتیب** مراحل درون بدنه‌ی یک تابع است، **پارامترها** همراه عدد ذخیره نمی‌شوند، دو اسکریپت را نمی‌توان **مقایسه** کرد، و مرحله‌ای که **نوع نادرست** داده را دریافت کند دیر یا هرگز شکست می‌خورد. این هفته یک دستورالعمل پاک‌سازی را به یک *شیء* تبدیل می‌کنیم: زنجیره‌ای مرتب از مراحل نوع‌دار که خودش را چاپ می‌کند، بر اساس مقدار مقایسه می‌شود، سیم‌کشی خودش را بررسی می‌کند و خودش را به‌صورت JSON ذخیره می‌کند.

مثال حل‌شده‌ی ۱: یک جهش در میانگین متحرک

در پرواز افقی نیروی ویژه‌ی عمودی $a_z \approx 10.35$ m/s² است ($n \approx 1.05$). یک نمونه ۵۰٫۳۳ m/s² می‌خواند، یعنی خطای تک‌نمونه‌ای حدود ۴۰+. میانگین مرکزی ۵ نمونه‌ای در هر یک از پنج موقعیتی که پنجره‌شان این نمونه را در بر می‌گیرد برابر است با

$$\bar{a}_z = \frac{4 \times 10.35 + 50.33}{5} = 18.35 \ \text{m/s}^2, \qquad n = \frac{18.35}{9.807} = 1.87.$$

میانگین‌گیری خطا را حذف نکرد؛ آن را روی پنج نمونه پخش کرد و به ۱٫۸۷g باورپذیر رساند، که از مانور واقعی ۱٫۸۰g بزرگ‌تر است. میانه‌ی ۷ نمونه با یک مقدار بد جابه‌جا نمی‌شود؛ پس اگر اول جهش حذف شود، همان مانور کشش بیشینه‌ی واقعی می‌ماند.

۲. کلاس پایه‌ی انتزاعی: قرارداد یک مرحله

یک *مرحله* داده می‌گیرد و داده‌ی جدید برمی‌گرداند. **کلاس پایه‌ی انتزاعی** (ABC) این قرارداد را در کد بیان می‌کند: کلاسی که از `abc.ABC` ارث می‌برد و `transform` را با `@abstractmethod` علامت می‌زند قابل نمونه‌سازی نیست، و هیچ زیرکلاسی هم که پیاده‌سازی آن را فراموش کند قابل نمونه‌سازی نیست. پایتون خطای `TypeError: Can't instantiate abstract class ... with abstract method transform` را هنگام *ساخت* شیء می‌دهد، که بسیار بهتر از یک `AttributeError` در میانه‌ی یک اجرای طولانی است. ABC می‌تواند **ویژگی انتزاعی** هم داشته باشد (`@property` روی `@abstractmethod`) تا هر زیرکلاس مجبور به ارائه‌ی یک مقدار شود، و متدهای **غیرانتزاعی** مشترک داشته باشد: در کتابخانه‌ی درس، `PipelineStage.__call__` نوع ورودی و خروجی و برازش‌شدن مرحله را برای همه‌ی مراحل بررسی می‌کند.

۳. dataclass: پیکربندی همان کلاس است

تنظیمات یک مرحله (ستون‌ها، طول پنجره، آستانه) هویت آن است: دو مرحله با تنظیمات یکسان، یک گام از دستورالعمل‌اند. دکوراتور `@dataclass` متدهای `__init__`، `__repr__` و `__eq__` را از ویژگی‌های دارای حاشیه‌نویسی نوع می‌نویسد؛ بنابراین تنظیمات برابر یعنی اشیای برابر، و چاپ شیء همه‌ی تنظیمات را نشان می‌دهد. چهار امکان مهم است:

- **`field(default_factory=list)`**. پیش‌فرض تغییرپذیری مثل `tags: list = []` هنگام تعریف کلاس با `ValueError` رد می‌شود، چون یک فهرست بین همه‌ی اشیا مشترک می‌شد (خطای فهرست مشترک هفته‌ی ۲). factory برای هر شیء فهرست تازه‌ای می‌سازد.
- **`frozen=True`**. مقداردهی به فیلد خطای `FrozenInstanceError` می‌دهد و شیء hashپذیر می‌شود. دستورالعملی که عددی منتشرشده را ساخته نباید بعداً تغییر کند.
- **`__post_init__`**. پس از `__init__` ساخته‌شده اجرا می‌شود؛ از آن برای اعتبارسنجی (پنجره‌ی مرکزی باید فرد و دست‌کم ۳ باشد) و یکسان‌سازی (فهرست ستون‌ها به tuple تبدیل می‌شود تا شیء hashپذیر بماند و فهرست JSON رفت‌وبرگشت کند) استفاده می‌کنیم. در کلاس frozen تنها نوشتن مجاز `object.__setattr__(self, "columns", tuple(columns))` است.
- **`field(init=False, repr=False, compare=False)`**. *حالت یادگرفته‌شده* را اعلام می‌کند: آرگومان سازنده نیست، چاپ نمی‌شود و در مقایسه شرکت نمی‌کند. `Standardize` مقدارهای یادگرفته‌شده‌ی `mean_` و `std_` را این‌گونه نگه می‌دارد؛ پس `Standardize(["x"])` برازش‌شده و برازش‌نشده برابرند: یک دستورالعمل در دو مرحله‌ی عمر. زیرخط پایانی قرارداد scikit-learn است که در هفته‌ی ۶ با آن آشنا می‌شویم.

۴. نوشتن دستی `__repr__`، `__eq__` و `__hash__`

`__repr__` برای برنامه‌نویس است و ترجیحاً کد معتبر پایتون (`eval(repr(x)) == x`)؛ `__str__` شکل کوتاهی است که `print` ترجیح می‌دهد. `__eq__` برای نوع نامرتبط `NotImplemented` برمی‌گرداند. تعریف `__eq__` بی‌صدا `__hash__ = None` قرار می‌دهد؛ `__hash__` باید دقیقاً همان داده‌ای را hash کند که `__eq__` مقایسه می‌کند و آن داده نباید تغییر کند. `DataPipeline`های مراحل را مقایسه می‌کند و عمداً hashناپذیر است، چون ممکن است مرحله‌ای برازش‌پذیر و تغییرپذیر داشته باشد.

۵. راهنمای نوع، جنریک‌ها و بررسی در زمان اجرا

راهنمای نوع قصد را مستند می‌کند و به ابزاری مثل `mypy` اجازه می‌دهد پیش از اجرا خطا پیدا کند، اما پایتون آن را اجبار نمی‌کند: `peak_g("9.8")` با راهنمای `np.ndarray` فقط درون بدنه شکست می‌خورد. `TypeVar` یک نوع جانگهدار است و `Generic[InT, OutT]` به کلاس اجازه می‌دهد پارامتر نوع بپذیرد؛ پس `PipelineStage[pd.DataFrame,` `np.ndarray]` درست مثل `list[int]` خوانده می‌شود. این پارامترها در زمان اجرا **پاک** می‌شوند، پس هر مرحله `INPUT_TYPE` و `OUTPUT_TYPE` را هم به‌صورت ویژگی `ClassVar` اعلام می‌کند. سازنده‌ی خط لوله هر جفت همسایه را با `issubclass(left.OUTPUT_TYPE, right.INPUT_TYPE)` بررسی می‌کند؛ `ToArray(...) >> RollingMean(...)` پیش از ورود هر داده‌ای با `TypeError` ی رد می‌شود که نام هر دو مرحله را می‌آورد.

۶. فیلتر Hampel

برای نمونه‌ی x_k و پنجره‌ی مرکزی w نمونه‌ای، m_k میانه‌ی پنجره و $\mathrm{MAD}_k = \mathrm{median}_j\ |x_j - m_k|$ میانه‌ی انحراف مطلق است. برای نوفه‌ی گاوسی با انحراف معیار σ، میانه‌ی $|x-\mu|$ برابر $\sigma\,\Phi^{-1}(3/4)\sigma = 0.6745\,\sigma$ است؛ پس $1.4826\,\mathrm{MAD}$ برآورد سازگاری از σ است که چند داده‌ی پرت نمی‌توانند آن را بزرگ کنند. فیلتر x_k را با m_k جایگزین می‌کند اگر

$$|x_k - m_k| > n_\sigma \cdot 1.4826 \cdot \mathrm{MAD}_k\,.$$

مثال حل‌شده‌ی ۲: فیلتر Hampel با دست

ارتفاع‌ها (متر): ۱۰۰٫۰، ۱۰۰٫۴، ۹۹٫۸، **۱۰۶٫۰**، ۱۰۰٫۳، ۱۰۰٫۱، ۹۹٫۹؛ $w = 5$ و $n_\sigma = 3$. در $k = 3$ پنجره‌ی {۱۰۰٫۴، ۹۹٫۸، ۱۰۶٫۰، ۱۰۰٫۳، ۱۰۰٫۱} پس از مرتب‌سازی ۹۹٫۸، ۱۰۰٫۱، **۱۰۰٫۳**، ۱۰۰٫۴، ۱۰۶٫۰ است؛ پس $m_3 = 100.3$. انحراف‌های ۰٫۱، ۰٫۵، ۵٫۷، ۰٫۰، ۰٫۲ پس از مرتب‌سازی ۰٫۰، ۰٫۱، **۰٫۲**، ۰٫۵، ۵٫۷ هستند؛ پس $\mathrm{MAD}_3 = 0.2$ و آستانه $3 \times 1.4826 \times 0.2 = 0.89$ متر است. چون $|106.0 - 100.3| = 5.7 > 0.89$، نمونه با ۱۰۰٫۳ جایگزین می‌شود. در $k = 4$ میانه ۱۰۰٫۱ است و $|100.3 - 100.1| = 0.2 < 0.89$: حفظ می‌شود. دو هشدار: جهشی به طول L نمونه‌ی پیاپی فقط وقتی حذف می‌شود که $w \geq 2L + 1$ (نمونه‌های سالم باید اکثریت باشند)؛ و با پنجره‌ی کوتاه روی نوفه‌ی خالص، خود MAD نوفه‌دار است و برخی نمونه‌های سالم هم با میانه‌ی محلی جایگزین می‌شوند (حدود ۴٫۸٪ نمونه‌های شتاب‌سنج با $w = 7$ در لاگ پرواز)، که یک هموارسازی ملایم است نه خطا.

مثال حل‌شده‌ی ۳: ترتیب مراحل مهم است

سیگنال $x = [1, 1, 1, 10, 1, 1, 1]$، فیلتر Hampel H با $w = 3$ و میانگین مرکزی ۳ نمونه‌ای S را در نظر بگیرید. **اول حذف جهش:** در $k = 3$ پنجره‌ی {۱، ۱۰، ۱} میانه‌ی ۱ و MAD صفر دارد، پس آستانه صفر است و ۱۰ با ۱ جایگزین می‌شود؛ $S(H(x))$ همه یک است و بیشینه ۱. **اول هموارسازی:** $S(x) = [1, 1, 4, 4, 4, 1, 1]$ چون $(1 + 1 + 10)/3 = 4$؛ اکنون میانه‌ی هر پنجره‌ی اطراف برآمدگی ۴ است، هیچ نمونه‌ای پرت شناخته نمی‌شود و بیشینه‌ی $H(S(x))$ برابر ۴ است. ترکیب جابه‌جاپذیر نیست، $H \circ S \neq S \circ H$؛ به همین دلیل خط لوله باید ترتیب را نگه دارد و `H >> S` و `S >> H` نابرابرند.

۷. کتابخانه‌ی درس: `aerospace_ai.pipeline`

فایل `src/aerospace_ai/pipeline.py` این اجزا را کنار هم می‌گذارد. کلاس `PipelineStage(ABC, Generic[InT, OutT])` متدهای `transform` (انتزاعی)، `fit` (بی‌اثر مگر مرحله یاد بگیرد)، `is_fitted`، `name`، `get_params` (فیلدهای dataclass با `init=True`)، `__call__` و `>>` را فراهم می‌کند و قلاب

`__init_subclass__` آن هر زیرکلاس را هنگام اجرای دستور class با نامش ثبت می‌کند. مراحل dataclass منجمد عبارت‌اند از `SensorToFrame` (هر `BaseSensor` به DataFrame)، `SelectColumns`، `DropNonFinite`، `FillGaps`، `HampelFilter`، `RollingMean`، `TimeDerivative` و `ToArray`؛ `Standardize` یاد می‌گیرد. `DataPipeline` یک tuple تغییرناپذیر از مراحل نگه می‌دارد، زنجیره را در `__init__` بررسی می‌کند، متدهای `__len__`، `__iter__`، `__getitem__` (برش یک زیرخط لوله برمی‌گرداند)، `__repr__`، `__str__` و `__eq__` را پیاده می‌کند، هر مرحله را روی خروجی مرحله‌ی قبل برازش می‌دهد، و `run` خروجی را همراه یک `StageReport` برای هر مرحله برمی‌گرداند (سطرهای ورودی و خروجی، تعداد NaN، خانه‌های تغییرکرده، زمان). `to_config` یک دیکشنری ساده برای `json.dumps` می‌سازد و `from_config` نام مراحل را در registry پیدا می‌کند و خط لوله‌ای برابر می‌سازد. به‌شکل خط لوله، تحلیلگر A برابر `RollingMean(5) >> SelectColumns(...)` و تحلیلگر B برابر `HampelFilter(7) >>` `RollingMean(5) >> SelectColumns(...)` است: متفاوت چاپ می‌شوند، نابرابرند، و JSON ذخیره‌شده‌ی B دقیقاً B را بازمی‌سازد.

خط لوله بر پایه‌ی `BaseSensor` نوشته شده، نه حسگرهای خاص؛ پس به کلاس‌های LiDAR و GPS تمرین ۱ وابسته نیست. حسگرهای خود دانشجویان بدون تغییر به `SensorToFrame` وصل می‌شوند و نسخه‌های مرجع پس از نمره‌دهی اضافه می‌شوند.

۸. حالت یادگرفته‌شده و نشت داده

مثال حل‌شده‌ی ۴: برازش فقط روی داده‌ی آموزشی

ارتفاع‌های آموزشی {۲، ۴، ۶} متر $\mu = 4$ و $\sigma = \sqrt{(4+0+4)/3} = 1.633$ متر می‌دهند؛ پس نمونه‌ی آزمون $x = 8$ متر به $z = (8-4)/1.633 = 2.449$ استاندارد می‌شود: کاملاً بیرون از بازه‌ی آموزش، و این دقیقاً همان چیزی است که مدل باید بداند. برازش روی داده‌ی آموزش **و** آزمون، {۲، ۴، ۶، ۸}، $\mu = 5$، $\sigma = \sqrt{(9+1+1+9)/4} = 2.236$ متر و $z = 1.342$ می‌دهد: نمونه‌ی آزمون اکنون عادی به نظر می‌رسد چون خودش در تعریف «عادی» شرکت کرده است. این نشت داده است، موضوع اصلی هفته‌ی ۵. در کد، `fit` فقط به داده‌ی آموزشی تعلق دارد و `DataPipeline.fit` هر مرحله را روی خروجی مراحل قبلیِ برازش‌شده برازش می‌دهد.

۹. پنجره‌ای از سطرها، پنجره‌ای از ثانیه‌ها نیست

خط لوله‌ی نرخ صعودی که ارتفاع بارومتری را روی ۵۱ *سطر* هموار و سپس مشتق‌گیری می‌کند، بیشینه‌ی ۲۰٫۹۹ m/s را در ثانیه‌ی ۹۹٫۷۶ گزارش می‌دهد، درست پیش از یک وقفه‌ی ۱٫۲ ثانیه‌ای در ثبت داده: نزدیک وقفه، پنجره‌ی ۵۱ سطری در یک طرف زمان بیشتری می‌پوشاند. درون‌یابی روی شبکه‌ی یکنواخت ۱۰ هرتز پیش از هموارسازی این اثر مصنوعی را حذف می‌کند (بیشینه‌ی ۱۹٫۹۱ m/s در ثانیه‌ی ۸۴٫۰). تمرین تمرینی ۳ همین مرحله‌ی `Resample` را همراه یک مرحله‌ی یادگیرنده‌ی `MinMaxScale` می‌سازد و کل خط لوله‌ی داده‌های هوایی را به‌صورت JSON ذخیره می‌کند.

آنچه اکنون می‌توانیم انجام دهیم

قرارداد را با ABC و پیکربندی را با dataclass منجمد بنویسیم، حالت یادگرفته‌شده را جدا کنیم، متدهای ویژه را درست بنویسیم، نوع را جایی که پایتون بررسی نمی‌کند بررسی کنیم و مراحل نوع‌دار را در خط لوله‌ای تکرارپذیر ترکیب کنیم. هفته‌ی بعد: الگوهای Observer، Strategy و Factory.